
Sensor Accuracy Worksheet

Cadence — an Ankle Freeze-of-Gait Monitor
for Parkinson's disease

ENGF0031 Scenario 2 — “Smart Clothing”

Group: 34

Date: June 5, 2026

Headline metric — sensitivity & specificity (not “accuracy”). Freeze episodes are rare relative to normal gait, so a single accuracy figure is dominated by the majority class and hides missed freezes. This worksheet therefore reports *sensitivity* (freezes correctly caught) and *specificity* (non-freezes correctly cleared) throughout.

Contents

Summary of results	2
1 Device and method overview	2
2 Task 1 — Test protocol design	3
3 Task 2 — Data collection	3
4 Task 3 — Data analysis	4
4.1 Defining “accuracy”: the confusion matrix	5
4.2 Choosing the threshold: a sensitivity/specificity sweep	5
4.3 A noisier session: where confounds bite	6
5 Task 4 — Limitations of the prototype	7
5.1 What features are present in the positive (freeze) readings?	7
5.2 How are these features detected in code?	7
5.3 Is there unwanted overlap that causes false positives?	7
5.4 How are false positives reduced in this first prototype (within a week)?	7
5.5 Better ways to reduce false positives, given more time	7
5.6 Are false negatives likely? How are they mitigated?	8
5.7 Summary: an error taxonomy	9
6 Task 5 — Final code	9
A Worked protocol example (provided with brief)	10
B Data-logging firmware (excerpt)	10
C On-board band-pass filters and calibration	11
D Offline analysis code (excerpt)	12
E Offline benchmark-model evaluation	12
References	13

Summary of results

Method / dataset	Sensitivity	Specificity	Notes
On-device Freeze-Index detector (capture 1, single clean session)	83%	100%	FI > 2.10 (clean optimum); deployed 1.815 → 83/92
On-device Freeze-Index detector (smoketest, longer noisy session)	80%	76%	threshold FI > 1.70; confound spikes
Offline CNN, public Daphnet bench- mark (subject-independent, LOSO)	71%	84%	$n = 8,982$ windows; honest cross-subject estimate

The device ships the on-board Freeze-Index detector with a deployed threshold of FI = 1.815 plus a movement-energy gate and two-window debounce (Sections 5 and 6). The single-session figure (83 / 100) is optimistic; the subject-independent Daphnet figure (71 / 84) is the honest estimate of how the method generalises to unseen wearers. Cadence is a **monitor**: it reports and logs freezes for a clinician and does *not* actuate any cue.

1 Device and method overview

Sensor and placement. A Circuit Playground Express (CPX) with its on-board LIS3DH accelerometer, worn at the *ankle* (lower shank) — the body site where freezing of gait (FoG) is most observable. Acceleration is sampled at 64 Hz on all three axes.

Detection principle. We use the clinically validated *Freeze Index* (Moore *et al.*, 2008): the ratio of high-frequency “freeze-band” power to low-frequency “locomotor-band” power of the accelerometer magnitude,

$$\text{FI} = \frac{P(3 \text{ Hz to } 8 \text{ Hz})}{P(0.5 \text{ Hz to } 3 \text{ Hz})}, \quad (1)$$

computed over a 4 s window (256 samples) every 2 s (128-sample hop). During a freeze the legs tremble at 3 Hz to 8 Hz while forward stepping (0.5 Hz to 3 Hz) collapses, so FI rises sharply. Because FI is a *ratio*, quiet standing (both bands near sensor noise) can also inflate it; we therefore gate the decision on a *movement-energy* floor (Bächlin *et al.*, 2010). The same mathematics runs on the board (fixed-point biquads) and offline in Python (Welch PSD), and the two were cross-checked.

Table 1: Signal-processing parameters (identical on-device and offline).

Parameter	Value	Reason
Sample rate	64 Hz	> 2× the 8 Hz freeze band; matches Daphnet
Window	256 samples (4 s)	several gait cycles for a stable PSD
Hop	128 samples (2 s)	a decision every 2 s
Locomotor band	0.5 Hz to 3 Hz	normal stepping cadence
Freeze band	3 Hz to 8 Hz	trembling-in-place during FoG
Deployed threshold	FI = 1.815	biased to sensitivity; gate controls false alarms

2 Task 1 — Test protocol design

The capture must let us (i) observe the *target* movement repeatedly, (ii) see *confounds* that could look similar, and (iii) find each segment easily in the data afterwards. We use a **scripted, auto-labelled protocol**: the firmware writes an incrementing *phase index* alongside every sample, so each movement block is unambiguously located in the CSV without any manual marking.

Table 2: Scripted capture protocol. The phase index is logged with every sample.

Phase	Activity	Duration	Purpose
0	Stand still	20 s	baseline noise / drift; gate calibration
1	Walk (normal)	40 s	dominant confound — rhythmic motion
2	Freeze (trembling in place)	20 s	the target event
3	Walk (normal)	40 s	repeat confound for consistency
4	Freeze	20 s	repeat target (consistency)
5	Walk	40 s	...
<i>(cycle continues: freeze/walk repeats, ending in a still block)</i>			

Design rationale.

- **Repeated targets.** Freeze appears multiple times (phases 2, 4, 6...) so we can check the detector fires *consistently*, not once by luck.
- **Built-in confound.** Walking is the hardest confound (it also has energy in the bands of interest), so it is interleaved around every freeze.
- **Baseline.** The opening still block measures resting sensor noise and is used to calibrate the movement-energy gate (Task 4).
- **Free / mixed movement.** A real walk includes turns, stops and transitions; the longer sessions (smoketest, ankle_run) include these.
- **Sensor position.** Ankle is fixed for all captures; a wrist variant was explored separately and is out of scope here.

A worked 20-minute protocol example (provided with the brief) is reproduced in Appendix A.

3 Task 2 — Data collection

Logging strategy. The CPX has no radio, so we use a *store-and-forward* logger: the board records to its 2 MB SPI flash while worn untethered on a LiPo battery, then replays the capture over USB afterwards. This is more robust than the worksheet’s serial-buffer example and removes the laptop from the walk entirely.

Sampling rate. The brief’s example logs every 200 ms (5 Hz). That is far too slow here: the freeze signature reaches 8 Hz, so by the Nyquist criterion we need at least 16 Hz and in practice much more to estimate band power cleanly. We log at 64 Hz, which also matches the Daphnet research dataset used for the offline model.

Memory budget (real). Each record is 7 bytes (`int16 x, y, z + uint8 phase`). At 64 Hz that is 448 B s^{-1} . The firmware pre-erases a 256 KB region, giving $256,000/448 \approx 9.5$ min per run; the full 2 MB flash holds 36 864 samples (confirmed by the device `STATUS` reply). UV or temperature, by contrast, change slowly and could be logged once per minute for weeks — sample rate must match the bandwidth of the phenomenon.

Table 3: Captures collected with this protocol (all real, on-body).

Capture	Duration	Samples	Rate	Role
capture 1	41.4 s	2652	64.0 Hz	clean, curated — threshold fitting
smoketest	91.3 s	5842	64.0 Hz	longer, noisier — confound stress test
ankle_run	161.0 s	10 305	64.0 Hz	second session — false-negative case

The data-logging firmware is listed in Appendix B.

4 Task 3 — Data analysis

Figure 1 shows capture 1: the accelerometer magnitude (top) with the four protocol phases shaded, and the Freeze Index (bottom). The FI is flat near 1 during standing and walking and spikes sharply during the freeze block — exactly the separation the detector relies on.

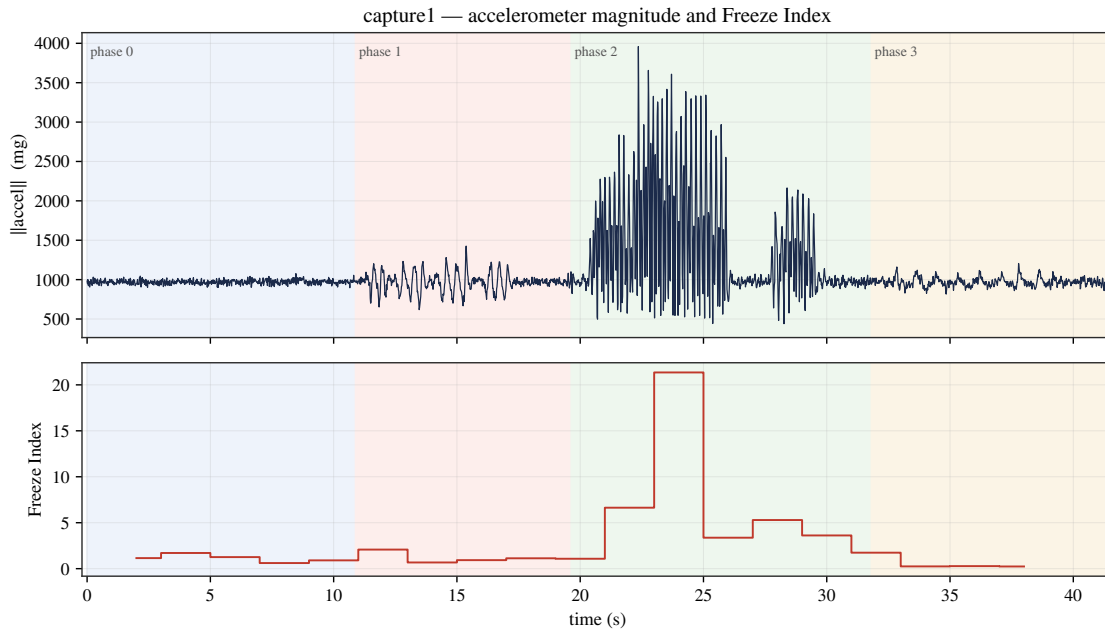


Figure 1: Capture 1 — accelerometer magnitude (top) and Freeze Index (bottom) with protocol phases shaded. The FI spikes only in phase 2 (freeze).

Table 4: Per-phase Freeze Index, capture 1 (4 s windows, 2 s hop).

Phase	Activity	n win.	FI mean	FI median	FI max
0	still	5	1.13	1.15	1.71
1	walk	4	1.20	1.03	2.08
2	freeze	6	6.89	4.45	21.35
3	walk	4	0.63	0.26	1.74

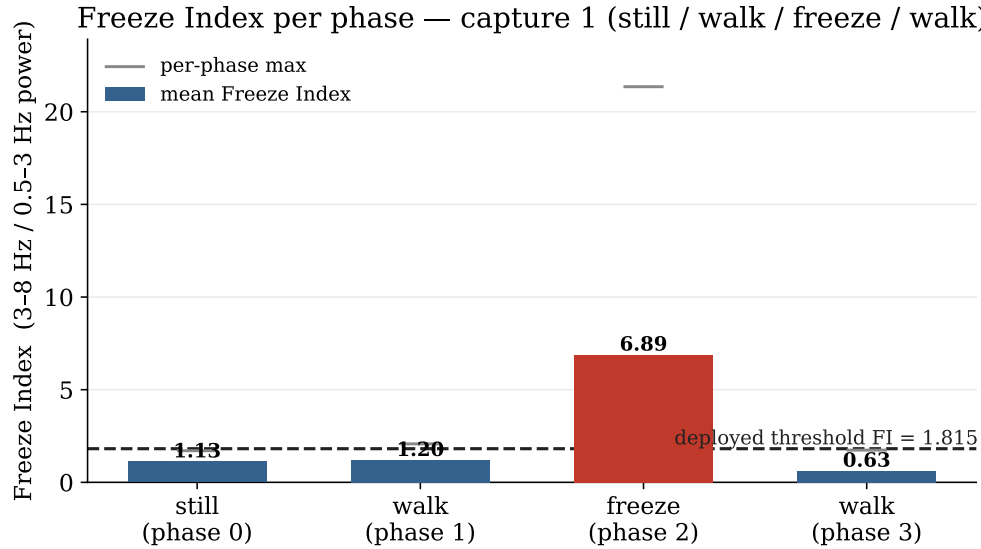


Figure 2: Mean Freeze Index per phase (capture 1) with the deployed decision threshold. The freeze phase sits far above every walking/standing phase; the grey ticks mark each phase’s maximum.

4.1 Defining “accuracy”: the confusion matrix

Every 2s decision is scored against its protocol label as one of four outcomes (Table 5). Writing TP, FP, TN, FN for the counts, the metrics this worksheet uses are

$$\begin{aligned} \text{sensitivity} &= \frac{TP}{TP + FN}, & \text{specificity} &= \frac{TN}{TN + FP}, \\ \text{precision} &= \frac{TP}{TP + FP}, & \text{accuracy} &= \frac{TP + TN}{TP + FP + TN + FN}. \end{aligned}$$

Table 5: The 2×2 confusion matrix. Columns are what the detector decides; rows are the ground truth from the protocol phase.

	Detector: freeze	Detector: no-freeze
Truly freeze	TP (<i>hit</i>)	FN (<i>missed freeze</i>)
Truly no-freeze	FP (<i>false alarm</i>)	TN (<i>correct clear</i>)

Why not a single “accuracy” number. Take the subject-independent CNN (Fig. 4, $n = 8982$ windows): $TN = 6842$, $FP = 1285$, $FN = 248$, $TP = 607$, giving

$$\text{sens} = \frac{607}{855} = 0.71, \quad \text{spec} = \frac{6842}{8127} = 0.84, \quad \text{prec} = \frac{607}{1892} = 0.32, \quad \text{acc} = \frac{7449}{8982} = 0.83.$$

The accuracy of 0.83 looks strong, but it is carried almost entirely by the 6842 easy true negatives. It hides that nearly a third of freezes are *missed* (sensitivity 0.71) and that only a third of freeze flags are *correct* (precision 0.32) — precisely the two failures a monitor is judged on. That is why the headline is sensitivity and specificity, not accuracy.

4.2 Choosing the threshold: a sensitivity/specificity sweep

The deployed cut-off is measured, not guessed. We swept the Freeze-Index threshold on capture 1, scoring all 19 windows against their phase label (6 freeze, 13 non-freeze). Table 6 gives the confusion counts and metrics at each threshold, with Youden’s $J = \text{sens} + \text{spec} - 1$ as a single balanced score.

Table 6: Freeze-Index threshold sweep on capture 1 (19 windows; 6 freeze, 13 non-freeze). J is maximised on the plateau from $FI = 2.10$ upward; the board deploys 1.815 (bold rows).

FI thr	TP	FP	TN	FN	sens	spec	acc	Youden J
1.00	6	6	7	0	1.00	0.54	0.68	0.54
1.50	5	3	10	1	0.83	0.77	0.79	0.60
1.815	5	1	12	1	0.83	0.92	0.89	0.76
2.10	5	0	13	1	0.83	1.00	0.95	0.83
3.00	5	0	13	1	0.83	1.00	0.95	0.83
4.00	3	0	13	3	0.50	1.00	0.84	0.50

Two things stand out. **(i) Accuracy is a misleading objective.** At $FI > 4.0$ the accuracy is still 0.84 — as high as anywhere sensible — yet sensitivity has collapsed to 0.50: half the freezes are missed. Optimising accuracy would push the threshold the *wrong* way, whereas Youden’s J (and the sensitivity/specificity pair) does not have this failure. **(ii) The deployed $FI = 1.815$ trades a little specificity for sensitivity.** The analysis optimum on this clean capture is the 2.10 plateau (sens 0.83, spec 1.00), but the board ships the lower 1.815 (sens 0.83, spec 0.92 here) so it stays sensitive on harder sessions, leaning on the movement-energy gate and debounce (Task 4) to suppress the single extra false positive this admits.

4.3 A noisier session: where confounds bite

Figure 3 shows the longer “smoketest” session (44 windows, 10 freeze). The freeze separation is still visible (freeze-phase FI median 3.49), but on this noisier capture several confound windows cross the threshold. At $FI > 1.70$, 8 of the 10 freezes are caught (sensitivity 0.80) while 8 of the 34 non-freeze windows also fire (specificity 0.76) — the worst reaching FI 10.23 in vigorous walking and FI 3.40 in quiet standing. These false positives, absent on the clean capture, are exactly what the movement-energy gate and debounce in Task 4 are built to remove.

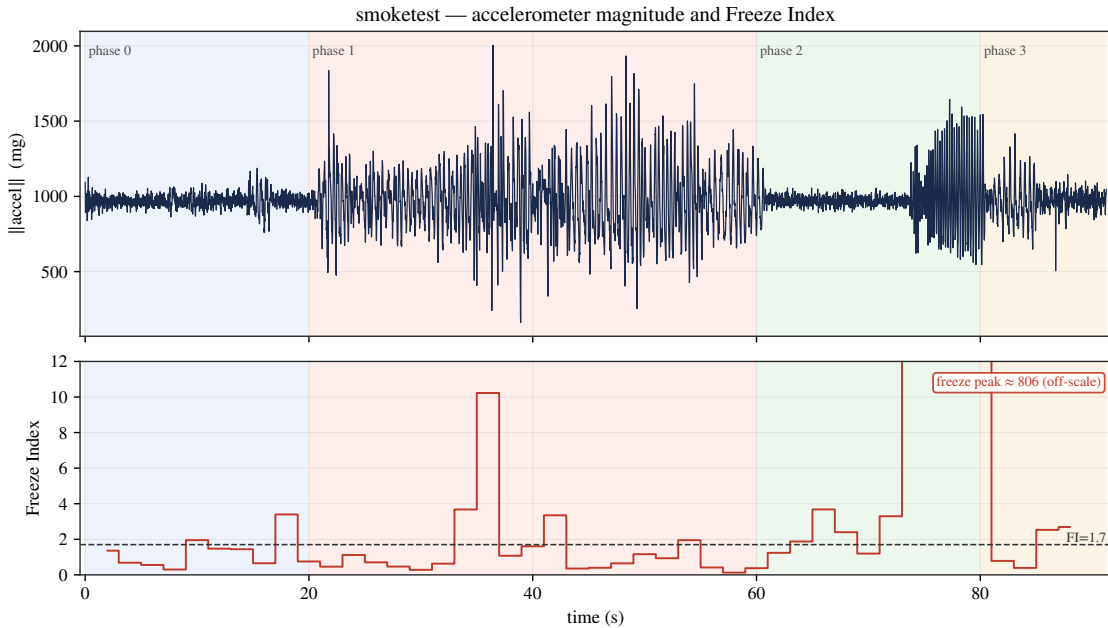


Figure 3: Smoketest (91s) — a longer, noisier session. The Freeze-Index axis is capped at 12 so the confound transients are visible: several walking and standing windows briefly cross the $FI = 1.70$ operating line (worst: walk 10.23, stand 3.40), while the genuine freeze (phase 2) runs far off-scale (peak ≈ 806). This separation is what makes the gate and debounce effective.

5 Task 4 — Limitations of the prototype

5.1 What features are present in the positive (freeze) readings?

A sustained burst of 3 Hz to 8 Hz oscillation on the accelerometer magnitude, *together with* the collapse of the 0.5 Hz to 3 Hz stepping rhythm. Numerically this is a large Freeze Index: capture 1 freeze FI mean 6.89 vs walk 1.20 and still 1.13 (Fig. 2); smoketest freeze FI median 3.49. The freeze band also carries high tremor power.

5.2 How are these features detected in code?

On the board, two cascaded second-order band-pass filters compute the freeze-band and locomotor-band power of each window; their ratio is the Freeze Index (`computeFI()`, Listing 1). A window is declared a freeze only if FI exceeds the threshold *and* the movement-energy gate passes. The identical ratio is computed offline with a Welch PSD (Appendix D) for scoring.

5.3 Is there unwanted overlap that causes false positives?

Yes. Vigorous walking transients and hand-handling of the board put transient energy into the freeze band, momentarily raising FI. Concretely, in the smoketest a standing window reached FI 3.40 and a walking window reached FI 10.23 — both well above the FI = 1.70 operating point, so both are false positives. This is exactly why specificity falls from 100 % on the clean capture (FI > 2.10) to 76 % on the noisier one (FI > 1.70).

5.4 How are false positives reduced in this first prototype (within a week)?

Three mitigations, all implemented on-device (Listing 1):

1. **Movement-energy gate.** Because FI is a ratio, quiet standing can make it explode from noise alone. We require the summed band power ($P_f + P_l$) to exceed a floor before any freeze is accepted; below the floor the state is forced to **STILL**. The floor is *calibrated per wearer* ($4 \times$ resting energy, clamped to 8 to 20).
2. **Two-window debounce.** A freeze must persist for `CONFIRM_ON` = 2 consecutive 2 s windows before it is confirmed, and clears after `CONFIRM_OFF` = 2 clear windows. Isolated single-window spikes (the smoketest false positives above) are rejected.
3. **Per-wearer calibration.** Button-A measures the resting floor at boot, so the gate adapts to the individual rather than using a fixed constant.

5.5 Better ways to reduce false positives, given more time

Replace the hand-tuned threshold with a *learned* model trained across many people. We trained a small convolutional network on the public **Daphnet** FoG benchmark and evaluated it **subject-independently** (Leave-One-Subject-Out: train on all but one patient, test on the held-out patient, repeat). Pooled over 8982 test windows the CNN scores **sensitivity 71 % and specificity 84 %** (Fig. 4), and it holds up — with spread — across individual held-out patients (Fig. 5). A complementary *interpretable* model (a random forest) lets us sweep the whole operating curve and expose the low-prevalence trade-off; that fuller benchmark evaluation, with the ROC, the precision-recall curve and the positive-predictive-value calculation, is in Appendix E.

Honest reading of the numbers. The CNN's 71 / 84 is *lower* than the prototype's single-session 83 / 100. That is the point: the single clean session *overstates* real-world accuracy, because the detector is tuned to one person on one day. The subject-independent figure is the trustworthy estimate, and the gap between them is the generalisation challenge a deployed device must close — with more data, per-wearer personalisation, and richer features.

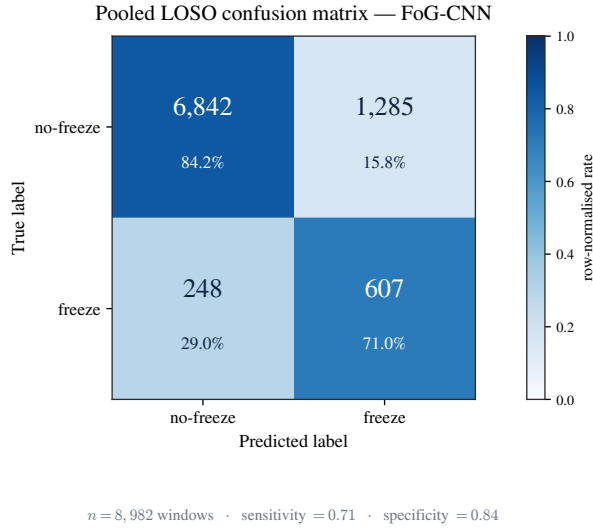


Figure 4: CNN, subject-independent (LOSO) on Daphnet — pooled confusion matrix: sensitivity 0.71, specificity 0.84, $n = 8,982$ windows.

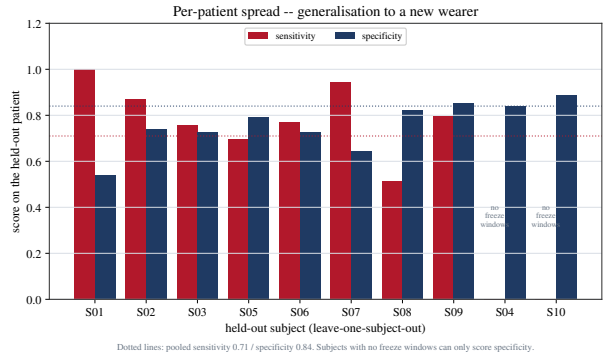


Figure 5: The same CNN, broken down per held-out patient. Sensitivity varies between individuals; two subjects (S04, S10) contributed no freeze episodes.

5.6 Are false negatives likely? How are they mitigated?

Yes — this is the more dangerous error for a monitor. An *akinetic* freeze that presents as a near-complete stop, with little trembling, has low power in *both* bands: FI stays low and the movement gate may even read it as “standing”. We observed exactly this in the `ankle_run` session (Fig. 6), where the freezes were performed as standing pauses rather than trembling-in-place: freeze-phase FI fell to 0.55 and 0.34 — *below* walking — and threshold scoring collapsed to 5% sensitivity. Mitigations:

- **Detect by absence of gait.** Add features that flag the sudden loss of the expected 0.5 Hz to 3 Hz stepping rhythm after walking, so an akinetic freeze is caught by what *stops* rather than what trembles.
- **Stride-time variability / cadence breakdown** as a complementary feature.
- **Lower the threshold** to raise sensitivity, explicitly trading away specificity — the operating point can be chosen along the ROC (Fig. 7) to suit a clinician’s preference.
- **Wearer coaching**, since freeze presentation varies between patients.

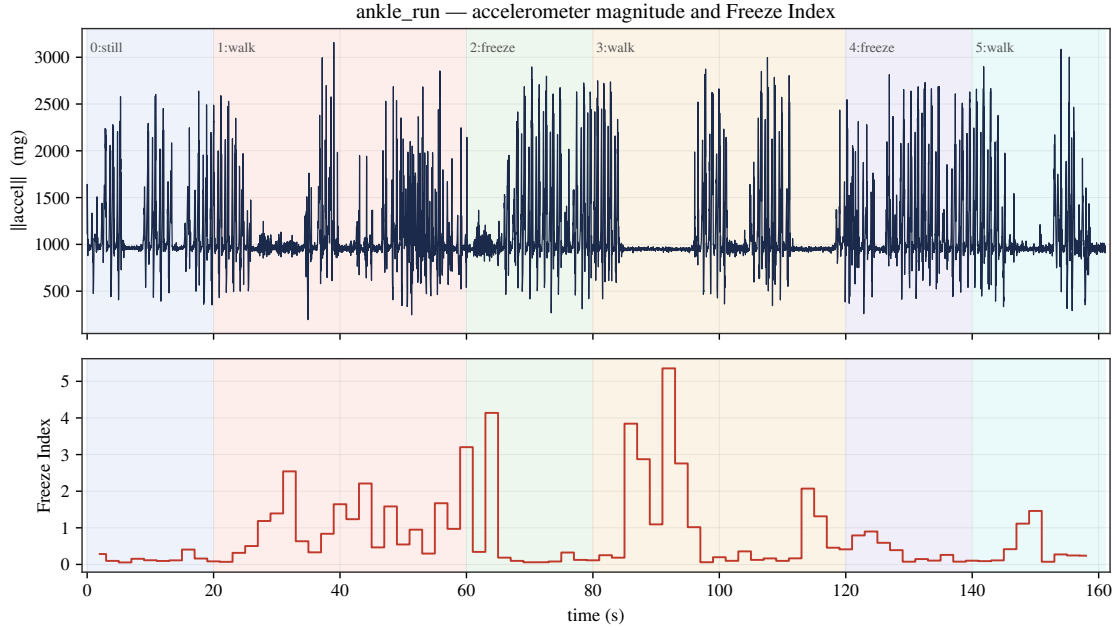


Figure 6: *ankle_run* — a false-negative case. The large accelerometer bursts fall in the *walk* phases; the Freeze Index stays low through both “freeze” phases (2 and 4), which were performed as standing pauses. The detector correctly cannot see a freeze that was not produced as a trembling-in-place event.

5.7 Summary: an error taxonomy

Table 7 consolidates the prototype’s failure modes, the real evidence for each, how it is handled within this one-week build, and what is left for future work.

Table 7: Error taxonomy for the on-board detector. “Status” is the state in this one-week prototype; FP = false positive, FN = false negative.

Error	Mechanism	Evidence	Mitigation	Status
FP — quiet standing	FI is a ratio: sensor-noise over sensor-noise can be large	gate rejects sub-floor energy	movement-energy gate ($P_f + P_l > \text{floor}$)	resolved
FP — walking transient	vigorous steps leak energy into the 3–8 Hz band	smoketest walk window FI 10.23	two-window debounce (CONFIRM_ON=2)	mostly resolved
FP — device handling	touching/adjusting the board adds broadband energy	—	gate + debounce; secure mounting	mostly resolved
FN — akinetic freeze	near-complete stop: low power in <i>both</i> bands	<i>ankle_run</i> freeze FI 0.55, 0.34	absence-of-gait feature; lower threshold	open (future)
FN — very short freeze	briefier than the two-window confirm time	—	shorten hop or CONFIRM_ON (costs FP)	trade-off

6 Task 5 — Final code

Listing 1 is the deployed on-board detector. The mitigations from Task 4 are highlighted in the comments: the **threshold** `FI_THRESHOLD = 1.815`, the **movement-energy gate** `last_energy > still_floor`, and the **two-window debounce** via `on_count / off_count`. The floor `still_floor` is set by per-wearer calibration (clamped 8 to 20).

```

1 const float FI_THRESHOLD = 1.815f; // freeze threshold (Task 3 sweep)
2 float still_floor = 12.0f; // movement-energy gate, per-wearer (8..20)
3 const float STILL_MARGIN = 4.0f; // floor = 4 x resting band-power
4 const uint8_t CONFIRM_ON = 2; // need 2 consecutive freeze windows
5 const uint8_t CONFIRM_OFF = 2; // need 2 consecutive clear windows
6
7 float computeFI() { // 4 s window -> Freeze Index
8     static float w[WINDOW];
9     float mean = 0.0f;
10    for (uint16_t i = 0; i < WINDOW; i++) { w[i] = ring[(head + i) % WINDOW]; mean += w[i]; }
11    mean /= WINDOW;
12    for (uint16_t i = 0; i < WINDOW; i++) w[i] -= mean; // remove gravity/DC
13    g_pf = runBand(FREEZE_SOS, w, WINDOW); // 3-8 Hz band power (biquad cascade)
14    g_pl = runBand(LOCO_SOS, w, WINDOW); // 0.5-3 Hz band power
15    return g_pf / (g_pl + 1e-9f); // Eq. (1): the Freeze Index
16 }
17
18 // --- every HOP (2 s): the gated, debounced decision -----
19 last_fi = computeFI();
20 last_energy = g_pf + g_pl; // movement energy (both bands)
21
22 bool moving = last_energy > still_floor; // GATE: reject quiet standing
23 bool freeze = moving && (last_fi > FI_THRESHOLD); // THRESHOLD on a moving wearer
24 g_state = !moving ? STILL : (freeze ? FREEZE : WALKING);
25
26 if (freeze) { on_count++; off_count = 0; } // DEBOUNCE: count run length
27 else { off_count++; on_count = 0; }
28 if (!cueing && on_count >= CONFIRM_ON) { cueing = true; freezeEpisodes++; }
29 else if (cueing && off_count >= CONFIRM_OFF) { cueing = false; } // confirmed clear

```

Listing 1: On-board freeze detector (Circuit Playground, cpx_fog_standalone.ino). The **threshold** and gate parameters fixed in Task 3 and the three Task 4 false-positive mitigations are **emphasised in red** and flagged in the inline comments: the movement-energy **gate** (`still_floor`), the **threshold** test (`FI_THRESHOLD`), and the two-window **debounce** (`on_count/off_count`).

The band-pass coefficients, the biquad cascade and the per-wearer calibration routine that produces `still_floor` are listed in Appendix C. The offline scoring uses the identical mathematics in Python (Appendix D); the two implementations were cross-checked so that the worksheet numbers and the on-body behaviour agree.

A Worked protocol example (provided with brief)

A structured 20-minute protocol for evaluating walking detection against confounds, adapted to the ankle FoG task:

Phase	Activity	Duration
1	Baseline (stand / sit still)	2 min
2	Normal walking	4 min
3	Fast walking	2 min
4	Confounds: arm-swing, shaking, sitting tasks, weight-shift	8 min
5	Mixed real-world (walk, stop, turn, pick up, sit-stand)	4 min

Key confounders to watch: arm-swinging while stationary, device handling, external vibration, and slow weight-shifting — each can mimic part of the gait signature and is a candidate false-positive source.

B Data-logging firmware (excerpt)

The store-and-forward logger writes one 7-byte record per sample at 64 Hz with an auto-incrementing phase label. NeoPixel colours guide the untethered wearer (cyan = still, green = walk, red =

freeze).

```

1 const Phase SCHED[] = { // segment index = the logged phase column
2   { "STILL", 20, false }, // 0
3   { "WALK", 40, false }, // 1
4   { "FREEZE", 20, true }, // 2
5   { "WALK", 40, false }, // 3
6   { "FREEZE", 20, true }, // 4 ... cycle repeats, ends on a STILL block
7 };
8
9 // steady 64 Hz sampling inside the RUNNING state:
10 float ax = CircuitPlayground.motionX(); // m/s^2
11 float ay = CircuitPlayground.motionY();
12 float az = CircuitPlayground.motionZ();
13 pushRecord((int16_t)(ax * MS2_TO_MG), // store as int16 milli-g
14            (int16_t)(ay * MS2_TO_MG),
15            (int16_t)(az * MS2_TO_MG), segIdx); // + the phase label

```

Listing 2: Scripted schedule and per-sample logging (cpx_fog_logger.ino).

C On-board band-pass filters and calibration

The Freeze Index needs the power in two frequency bands. On the board each band is a second-order Butterworth band-pass realised as two cascaded second-order sections (coefficients exported from the offline design at 64 Hz) and applied as a Direct-Form-II-transposed biquad cascade. `runBand()` returns $\sum y^2$ over the window, which is proportional to band power; the proportionality constant cancels in the FI ratio (Eq. 1).

```

1 struct Sos { float b0, b1, b2, a1, a2; }; // a0 normalised to 1
2 const uint8_t N_SOS = 2; // order-2 Butterworth -> 2 sections
3
4 const Sos FREEZE_SOS[N_SOS] = { // 3-8 Hz freeze band
5   { 0.04427971f, 0.08855942f, 0.04427971f, -1.24067070f, 0.62897179f },
6   { 1.00000000f, -2.00000000f, 1.00000000f, -1.69751871f, 0.79495806f },
7 };
8 const Sos LOCO_SOS[N_SOS] = { // 0.5-3 Hz locomotor band
9   { 0.01278734f, 0.02557469f, 0.01278734f, -1.69059402f, 0.75072617f },
10  { 1.00000000f, -2.00000000f, 1.00000000f, -1.93848688f, 0.94143123f },
11 };
12
13 // Run one band through the cascade; return sum(y^2) over the window (= band power).
14 float runBand(const Sos *sos, const float *x, uint16_t n) {
15   float z1[N_SOS] = {0}, z2[N_SOS] = {0}, sumsq = 0.0f;
16   for (uint16_t i = 0; i < n; i++) {
17     float in = x[i];
18     for (uint8_t s = 0; s < N_SOS; s++) { // direct-form-II transposed
19       float out = sos[s].b0 * in + z1[s];
20       z1[s] = sos[s].b1 * in - sos[s].a1 * out + z2[s];
21       z2[s] = sos[s].b2 * in - sos[s].a2 * out;
22       in = out;
23     }
24     sumsq += in * in;
25   }
26   return sumsq; // proportionality cancels in FI ratio
27 }

```

Listing 3: Deployed band-pass coefficients and the biquad cascade (cpx_fog_standalone.ino).

Per-wearer calibration of the gate. Pressing button A while standing still measures the resting band-power over one window and sets the movement-energy floor to `STILL_MARGIN` $\times$ resting, clamped to 8 to 20. The clamp band comes from bench measurement: a dead-still board on this hardware reads energy ≈ 2.5 (up to ~ 5 with handling transients) while *any* deliberate motion drives it into the tens-to-hundreds, so any floor in 8 to 20 sits safely above still-noise yet well below real gait.

```

1 const float STILL_MARGIN = 4.0f; // floor = margin x resting energy
2 const float MIN_FLOOR = 8.0f; // stay ABOVE measured still-noise (~5)
3 const float MAX_FLOOR = 20.0f; // ... yet well BELOW real-motion energy (tens+)
4

```

```

5 if (calib_left == 0 && filled >= WINDOW) {
6   computeFI(); // fills g_pf, g_pl for the still window
7   float rest = g_pf + g_pl; // resting movement energy (both bands)
8   still_floor = STILL_MARGIN * rest; // gate threshold for THIS wearer + board
9   if (still_floor < MIN_FLOOR) still_floor = MIN_FLOOR;
10  if (still_floor > MAX_FLOOR) still_floor = MAX_FLOOR;
11 }

```

Listing 4: Button-A calibration of the movement-energy floor (cpx_fog_standalone.ino).

D Offline analysis code (excerpt)

The Python reference implementation (fog/dsp.py) that scores the captures. It is the same Freeze Index and movement-energy used on the board, validated against the firmware output.

```

1 def magnitude(window): # (T,3) accel -> (T,) mean-removed |a|
2     mag = np.linalg.norm(window, axis=1)
3     return mag - mag.mean() # strip gravity / DC
4
5 def band_power(sigld, fs, band): # power in [lo,hi) Hz via Welch PSD
6     f, pxx = signal.welch(sigld, fs=fs, nperseg=min(len(sigld), 256))
7     lo, hi = band
8     mask = (f >= lo) & (f < hi)
9     df = float(f[1] - f[0])
10    return float(np.sum(ppx[mask]) * df)
11
12 def freeze_index(window, fs=64): # Eq. (1): Moore et al. 2008
13     mag = magnitude(window)
14     loco = band_power(mag, fs, (0.5, 3.0)) # locomotor band
15     return band_power(mag, fs, (3.0, 8.0)) / (loco + 1e-9) # freeze / loco
16
17 def movement_energy(window, fs=64): # gate signal: total band power
18     mag = magnitude(window)
19     return band_power(mag, fs, (0.5, 3.0)) + band_power(mag, fs, (3.0, 8.0))

```

Listing 5: Freeze Index and movement-energy (fog/dsp.py).

Reproducibility. Every figure and number in this worksheet is produced from the captures in `accuracy_figs/captures/` by `analyze_worksheet.py`; e.g. capture 1 is scored with `python analyze_worksheet.py accuracy_figs/captures/capture1.csv --freeze-phases 2`.

E Offline benchmark-model evaluation

To estimate how a *learned* detector would generalise beyond our own captures, two models were trained on the public **Daphnet** Freezing-of-Gait dataset (Bächlin *et al.*, 2010) and evaluated **subject-independently** with *Leave-One-Subject-Out* (LOSO) cross-validation: train on every patient but one, test on the held-out patient, and repeat so that every prediction is made on a wearer the model never saw in training. Evaluating on the same person you trained on flatters the result; LOSO is the honest estimate of field performance.

Model 1 — convolutional network (headline cross-subject estimate). A small 1-D CNN over the windowed accelerometer signal. Pooling all held-out predictions (8982 windows) gives sensitivity 0.71 and specificity 0.84 (Fig. 4); the per-patient breakdown (Fig. 5) shows the spread between individuals — some patients freeze in ways the pooled model catches easily, others much less so.

Model 2 — interpretable random forest (operating curve and prevalence). A random forest over hand-engineered band-power features. Because it emits a graded score, every decision threshold can be swept to draw the full ROC (Fig. 7, AUC 0.82) and precision-recall curve (Fig. 8). The ROC lets a clinician choose the sensitivity/specificity trade-off explicitly rather than accept one fixed operating point.

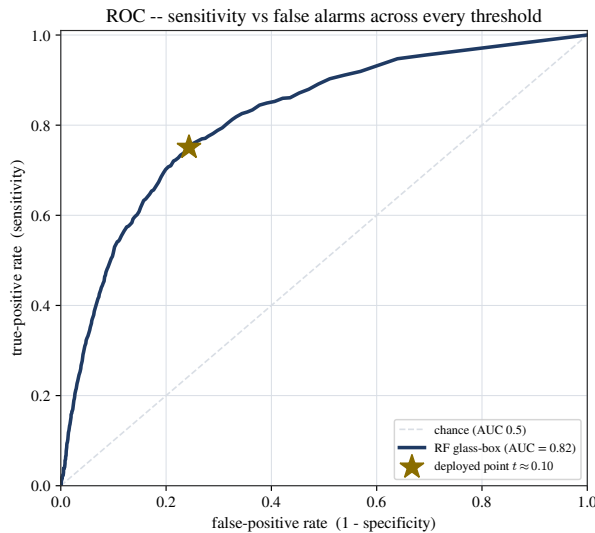


Figure 7: ROC of the interpretable model across all thresholds (AUC 0.82). Each point is a sensitivity/specificity trade-off a clinician could select.

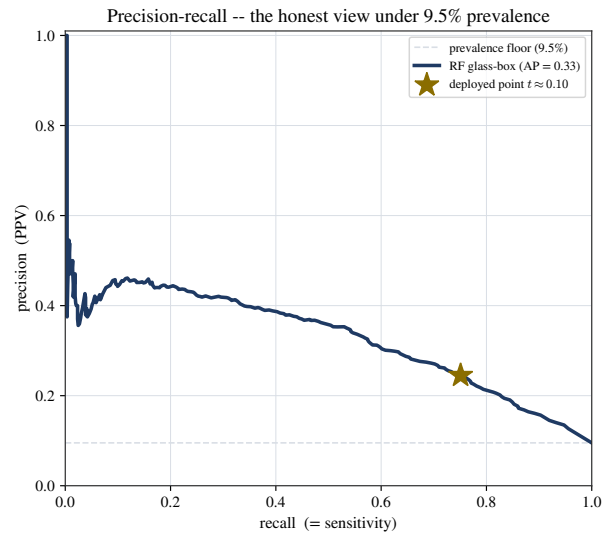


Figure 8: Precision–recall for the same model. Average precision is 0.33 at the dataset’s 9.5 % freeze prevalence — the realistic view.

Why precision matters more than “accuracy” here. Freezes are only about **9.5 %** of the Daphnet windows. At that prevalence even a good detector has modest *precision*. With sensitivity 0.71 and specificity 0.84 the positive predictive value is

$$\text{PPV} = \frac{0.71 \times 0.095}{0.71 \times 0.095 + (1 - 0.84) \times 0.905} \approx 0.32,$$

so only about a third of the windows flagged as a freeze are truly freezes — and the random forest’s average precision (0.33) tells the same story. A bare “accuracy” figure would read ~ 0.83 and hide this completely. This is exactly why the worksheet reports sensitivity and specificity (and here precision/PPV) rather than accuracy, and why Cadence is positioned as a **monitor** whose flags a clinician reviews — not an alarm that must be right every time.

References

- [1] S. T. Moore, H. G. MacDougall, W. G. Ondo. “Ambulatory monitoring of freezing of gait in Parkinson’s disease.” *Journal of Neuroscience Methods*, 167(2):340–348, 2008. (*Defines the Freeze Index as the 3–8 Hz / 0.5–3 Hz accelerometer power ratio.*)
- [2] M. Bächlin, M. Plotnik, D. Roggen, I. Maidan, J. M. Hausdorff, N. Giladi, G. Tröster. “Wearable assistant for Parkinson’s disease patients with the freezing of gait symptom.” *IEEE Transactions on Information Technology in Biomedicine*, 14(2):436–446, 2010. (*Movement-energy gating; source of the Daphnet dataset.*)
- [3] Daphnet Freezing of Gait dataset, UCI Machine Learning Repository. Tri-axial accelerometer recordings (ankle, thigh, trunk) from Parkinson’s patients at 64 Hz, with freeze annotations.